第12天 · 保存状态，让任务能够继续

检查点记录做到哪里，恢复时先确认还是同一项任务。

标准路线：60分钟；忙碌日：20分钟；扩展练习另计。

今天的目标

- 区分当前状态、检查点与长期记忆。

- 实际观察首次运行与重复恢复的不同。

- 验证问题或语料改变时不会误用旧检查点。

开始之前

- 完成第11课，或能在practice目录运行日课命令。

- 只使用教学材料；修改配置前保留原值，实验结束后恢复或记下修改。

一小时安排

- 15分钟 · 理解概念：阅读四节讲解，记下今天最关键的一个区别；外部资料按指定小节选读。

- 30分钟 · 操作与修改：先完成四步带练，再改变一个参数，保留修改前后的结果。

- 10分钟 · 验收与自测：对照三项验收，完成两道自测并阅读解析。

- 5分钟 · 记录与收尾：写下结果、未解决的问题和下次打开项目后的第一步。

01 · 理解概念

状态描述的是这一次任务

状态包含当前问题、已取得的证据、执行阶段和待完成步骤。它像实验记录中的当前进度，告诉程序接下来可以做什么。它不等于用户长期偏好，也不意味着模型已经通过这些记录重新训练。

本课先检索教学记录，把证据ID保存到outputs/checkpoint.json，再从保存的ID读取材料并组装答案。恢复时继续使用已保存的检索结果，而不是只打印一句“已恢复”却重新执行所有阶段。

检查点要有身份与版本

一个可用检查点至少需要说明属于哪个问题、语料是什么版本、当前阶段和已完成结果。课程文件记录schema_version、question、corpus_sha256、phase与evidence_ids，使程序能判断它是否适用于当前任务。

仅仅发现一个同名文件是不够的。如果用户换了问题，或资料内容已经变化，旧证据可能不再适合。本课只有问题和语料摘要同时一致，且resume允许恢复，才复用已有记录；否则建立本次的新检查点。

恢复成功与最终正确分别验收

loaded_existing表示是否读取了适用的已有检查点。checkpoint.phase描述保存时的阶段，state.phase描述本次继续执行后的阶段。二者不同是正常情况：从retrieved阶段恢复后，可以到completed阶段。

恢复说明控制流程能够继续，不自动证明证据正确、引用充足或研究结论可靠。还要按前面课程检查材料与回答。对于会发送消息、扣费或写外部系统的动作，恢复还涉及重复执行问题，第17课再学习批准与幂等。

用主动重跑理解恢复条件

先将resume设为false运行，得到明确的新起点。再设回true，用完全相同的问题运行，观察loaded_existing成为true。最后只改问题再运行，验证旧检查点没有被错误复用。

不需要删除整套outputs，也不需要强行中断电脑。这个小实验就能演示保存、匹配与恢复；如果检查点文件损坏，应报告格式问题并从新任务重新建立，而不是悄悄接受半份状态。

02 · 一步一步做

命令在解压后的 practice 文件夹执行。Windows可使用 .\.venv\Scripts\python.exe；macOS/Linux可使用 .venv/bin/python 替代 python。

1. 建立新检查点

为什么做：让首次与复用状态可区分。

把day12.resume设为false，保留默认question并执行；这会重新写本课检查点。

```
python run.py day 12
```

预期结果：result.loaded_existing为false，生成outputs/checkpoint.json并完成本次组装。

2. 查看真正保存的内容

为什么做：核对恢复所依据的数据。

检查文件中的question、corpus_sha256、phase与evidence_ids。

```
python -m json.tool outputs/checkpoint.json
```

预期结果：保存的是检索阶段证据ID及任务身份，不是模型训练参数。

3. 恢复相同任务

为什么做：验证程序确实复用已完成阶段。

将day12.resume改回true，不改变question，再运行。

```
python run.py day 12
```

预期结果：loaded_existing为true；state.resumed_from为retrieved，state.phase为completed。

4. 给旧检查点一个新问题

为什么做：确认任务身份检查生效。

只把day12.question改成What does relative abundance measure?，再运行并比较；最后恢复原问题，按需重建检查点。

```
python run.py day 12
```

预期结果：新问题不复用旧问题的检查点；loaded_existing为false，证据随任务更新。

改一改 · 只改变resume，观察是否复用

- 固定question和语料，设置day12.resume=false运行，记录loaded_existing。

- 仅将resume改为true再运行，比较loaded_existing与保存的evidence_ids。

- 说明第一次是建立检查点，第二次是从适用的检查点继续；保留两次结果。

观察什么：同一问题与语料下，关闭恢复时建立新记录，开启恢复后可以复用；答案相似不等于两次走了完全相同路径。

机制解释：恢复机制需要可以观察的身份匹配与状态转移，不能只依靠最终答案判断。

常见问题

第一次就显示loaded_existing=true

原因：此前运行留下了适用于同一问题与语料的检查点。

处理：先用resume=false运行一次，再开启恢复；不要把已有记录误认为程序状态错误。

恢复时JSON解析失败

原因：检查点被手工编辑成了无效JSON或文件不完整。

处理：先保留错误记录供排查，将resume设为false重新建立；不要在不知道字段含义时随意修改状态。

03 • 验收

第12课运行结果、一次参数对照，以及自己能解释的学习笔记。

成果文件：outputs/day12.json

☐ 能够找到并解释检查点的任务身份和语料摘要。

☐ 完成resume=false与true的可复现对照。

☐ 通过修改问题确认旧检查点不会被误用。

自测与解析

1. 只要outputs/checkpoint.json存在，是否就应该使用?

- A. 是，文件名相同即可

- B. 是，任何问题都能共享证据

- C. 否，还要核对问题、语料版本与恢复条件

- D. 否，状态永远不能持久保存

答案：C。 存在文件仅说明曾经保存过；任务身份与输入版本匹配才能合理复用。

2. 保存检查点后，模型发生了什么变化?

- A. 自动完成了再训练

- B. 获得了永久记忆能力

- C. 模型权重必然改变

- D. 程序保存了任务状态，不能据此推断模型被训练

答案：D。 检查点是应用层记录；是否训练模型是另一套过程。

小词典

- State：当前任务输入、阶段和中间结果的集合。

- Checkpoint：保存下来供后续匹配和恢复的状态。

- 任务身份：区分当前任务与其他任务的输入及版本信息。

04 · 我的记录

- 当前检查点足以解释为什么恢复到这个阶段吗?

- 如果未来工具具有外部写入动作，重跑时需要额外防止什么?

忙碌日：20分钟

- 用3分钟读“状态描述的是这一次任务”，写下一个关键区别。

- 用12分钟运行 python run.py day 12，打开 outputs/day12.json 并核对第一项预期结果。

- 用5分钟完成第一道自测并写一句解释；完整参数实验和其余验收留待下次。

留下本课运行结果与一句自己的解释；精简完成单独记录，不等于完整课程已经验收。

有余力再做：20分钟选学 · 设计一个检查点升级方案

- 假设下一版增加required_evidence字段，写出旧检查点缺字段时如何处理。

- 说明何时可以迁移，何时应重新运行。

- 给恢复失败写一条清楚的提示，并保留原始错误便于排查。

参考资料与阅读范围

BioAgentLab：状态与记忆：只读状态、会话和长期记忆的不同用途，以及生命周期。

https://github.com/zbyloveyj/BioAgentLab/blob/main/book/zh/05_state_and_memory.md

Python 官方：JSON 编码与解码：只读基本读写、Python/JSON类型对应和格式错误说明。

https://docs.python.org/3/library/json.html

下一步

下一课处理跨任务记忆，加入用户归属、过期和删除。